

DART LANGUAGE BASICS

Functions in Dart

A visual, no-boring-parts guide to declaring, calling,
and mastering functions in Dart

PREPARED FOR STUDY • DART SDK • 2026



1 What is a Function?

The building block of reusable Dart code

A **function** is a named, reusable block of code that performs a task. Instead of repeating the same logic everywhere, you wrap it once and simply *call* it whenever you need it. In Dart, functions are so flexible that they can even be treated like values — stored in variables, passed as arguments, and returned from other functions.

Anatomy of a Dart Function

Return type

Parameter list

```
int calculateArea (int width, int height) {  
    return width * height;  
}
```

```
} // closing brace ends the function body
```

🔁 Reusable

Write logic once, call it as many times as you want from anywhere in your code.

📦 First-class

Functions can be assigned to variables and passed around just like `int` or `String`.

🔗 Typed

Dart is statically typed — parameters and return values usually declare their type.

Quick fact: Every Dart app starts by calling a special function named `main()` — it's the entry point of the program.

2 Three Ways to Write a Function

Same idea, different syntax flavors

① Named Function (classic)

```
int square(int n) {  
  return n * n;  
}
```

② Arrow Function (short one-liners)

```
int square(int n) => n * n;  
// same result, no curly braces, no "return" keyword
```

③ Anonymous Function (no name, used inline)

```
var numbers = [1, 2, 3, 4];  
numbers.forEach((n) {  
  print('Number: $n');  
});
```

Named

Best for logic you'll reuse in many places.

e.g. square(), main()

Arrow (=>)

Best for short, single-expression logic.

e.g. isEven(n) => n % 2 == 0;

Anonymous

Best for one-off logic passed directly as an arg.

e.g. list.map((x) => x*2)

3 Parameters, Decoded

Dart gives you four flexible ways to pass data in

Required Positional

```
greet(String name) { ... }  
greet('Nazer');
```

Optional Positional []

```
greet(String n,[String? t]){}  
greet('Nazer', 'Sir');
```

Named { }

```
greet({required String n}){}  
greet(n: 'Nazer');
```

Default Values

```
greet({String t = 'Hi'}){}  
greet(); // uses 'Hi'
```

Style	Syntax	Must you pass it?	Call example
Required positional	(Type name)	Yes, in order	fn('Nazer')
Optional positional	([Type name])	No	fn('Nazer','Bisaya')
Named	({required Type name})	Yes, by name	fn(name: 'Nazer')
Named + default	({Type name = value})	No — falls back	fn()

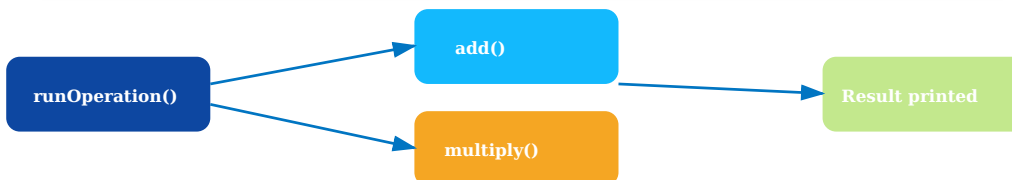
Tip: Named parameters make long function calls self-documenting — great for widgets and configuration-heavy code, which is why Flutter uses them everywhere.

4 Functions as Values

Higher-order functions & closures

Passing a function into another function

```
void runOperation(int a, int b, Function op) {  
  print(op(a, b));  
}  
  
int add(int x, int y) => x + y;  
  
runOperation(4, 5, add); // prints 9
```



`runOperation()` doesn't care *which* function you hand it — that's the power of treating functions as values.

Closures — a function that remembers

```
Function makeCounter() {  
  int count = 0;  
  return () {  
    count++;  
    print(count);  
  };  
}  
  
var counter = makeCounter();  
counter(); // 1  
counter(); // 2 — remembers "count" between calls!
```

Why it matters: A closure "closes over" the variables around it, keeping them alive even after the outer function has finished running. This powers things like counters, caches, and Flutter state callbacks.

5 Recursion in Action

A function that calls itself

```
int factorial(int n) {  
  if (n <= 1) return 1;  
  return n * factorial(n - 1);  
}  
factorial(4); // 24
```

factorial(4) → factorial(3) → factorial(2) → factorial(1) → 1

1 → 2×1=2 → 3×2=6 → 4×6=24

Rule of thumb: Every recursive function needs a **base case** (where it stops) — otherwise it calls itself forever and crashes with a stack overflow.

Cheat Sheet

Concept	One-line meaning
void function	Runs code but returns nothing
Arrow function	Shorthand for a single-expression body
Optional parameter	Can be skipped when calling
Named parameter	Passed using name: value syntax
Anonymous function	A function with no name, used inline
Higher-order function	Takes or returns another function
Closure	Remembers variables from its enclosing scope
Recursion	A function calling itself, with a base case